Informe de Sprint 3

Antonio de Benito, Pablo Berástegui y Gonzalo García

Período: 01/02/2026 – 21/02/2026

Proyecto

TradeData – Análisis de Datos de Trading de Criptomonedas

1. Objetivo del Sprint

En este sprint realizamos diferentes transformaciones sobre nuestros datos para poder consultar de forma sencilla *insights* sobre los mismos. En primer lugar, llevamos a cabo una transformación de los datos a formato `Parquet` y, posteriormente, extraemos diferentes indicadores técnicos (KPIs) sobre nuestra criptomoneda.

El objetivo principal es estructurar la información en tres capas (Bronce, Plata y Oro) y facilitar su posterior análisis mediante herramientas como AWS Glue, Apache Spark y Athena.

Se puede observar todo el código en este repositorio de GitHub.

2. Historias de Usuario Incluidas

2.1. Historia de usuario 3

Como: Manager del proyecto
Quiero: Que las consultoras realicen una implementación del almacenamiento de datos históricos basada en tres capas, separadas en carpetas distintas:

- `bronze/`
- `silver/`
- `gold/`

Además, deberán:

- Utilizar AWS Glue y Apache Spark para leer los ficheros en formato CSV almacenados en la capa Bronce y guardarlos en la capa Plata en formato Parquet.
- Calcular los siguientes indicadores (KPIs) para realizar un análisis técnico a partir de la información almacenada en la capa Plata:

- Moving Average Simple (SMA 200)

$$SMA_{200}(t) = \frac{1}{200}\sum_{i=0}^{199} P_{t-i}$$

- Moving Average Exponential (EMA 50)

$$EMA_n(t) = \alpha P_t + (1-\alpha)EMA_n(t-1)$$

- Relative Strength Index (RSI)

$$RS_t = \frac{AvgGain_{14}(t)}{AvgLoss_{14}(t)}$$

$$RSI_{14}(t) = 100 - \frac{100}{1+RS_t}$$

- Moving Average Convergence Divergence (MACD)

$$MACD(t) = EMA_{12}(t) - EMA_{26}(t)$$

- Almacenar la información de estos indicadores en la capa Oro.

Para: Realizar un análisis técnico de las criptomonedas.

2.1.1. Análisis

Para conseguir el objetivo planteado, seguimos una arquitectura basada en tres capas dentro de un único bucket de S3. Mantenemos un único bucket, ya que las distintas tablas estarán relacionadas y, de esta manera, solo es necesario ejecutar un único crawler para catalogar todas las capas.

Dentro del bucket se crean tres carpetas diferenciadas:

- `bronze/`
- `silver/`
- `gold/`

Esta organización permite separar datos crudos, datos transformados y datos enriquecidos con indicadores técnicos, facilitando la gobernanza y el análisis posterior.

2.1.2. Implementación

En primer lugar, pasamos de la capa Bronce (creada en el sprint anterior) a la capa Plata. Para ello, accedemos a los datos mediante el Glue Data Catalog (configurado previamente) y los convertimos a formato Parquet, almacenándolos en la carpeta correspondiente a la capa Plata. Todo este proceso se gestiona mediante un job de AWS Glue.

Este es el código usado para pasar de bronce a plata (**`bronze2silver.py`**).

```
import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job

args = getResolvedOptions(sys.argv, ['JOB_NAME'])
sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args['JOB_NAME'], args)

dynamic_frame = glueContext.create_dynamic_frame.from_catalog(
    database="trade_data_eth_imat3a03",
    table_name="eth_datos_grandes_eth_project"
)

dynamic_frame = DynamicFrame.fromDF(
    dynamic_frame.toDF().drop("partition_0"),
    glueContext,
    "dynamic_frame"
)

glueContext.write_dynamic_frame.from_options(
    frame=dynamic_frame,
    connection_type="s3",
    connection_options={"path": "s3://datos-grandes-eth-project/silver/"},
    format="parquet"
)

job.commit()
```

Posteriormente, utilizando otro job de AWS Glue, leemos los datos desde el Data Catalog y calculamos cada una de las métricas técnicas usando Apache Spark.

Y este es el código usado para pasar de plata a oro (**silver2gold.py**).

```
from pyspark.sql import SparkSession, functions as F
from pyspark.sql.window import Window
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job

sc = SparkContext()
glueContext = GlueContext(sc)

INPUT_PATH = "s3://datos-grandes-eth-project/silver/"
OUTPUT_PATH = "s3://datos-grandes-eth-project/gold/"
```

```
ETH_SYMBOL = "BINANCE:ETHUSDT"

K_EMA50 = 300
K_EMA12 = 200
K_EMA26 = 300
K_EMA9  = 120

def ordered_history_array(value_col: str, order_col: str, w: Window) -> F.
    Column:
    return F.expr(
        f"transform(sort_array(collect_list(named_struct('k', {order_col}, 'v',
     {value_col})) over w), x -> x.v)"
    )

def ema_truncated_from_array(arr_col: str, span: int) -> F.Column:
    alpha = 2.0 / (span + 1.0)
    decay = 1.0 - alpha

    return F.expr(f"""
      CASE
        WHEN {arr_col} IS NULL OR size({arr_col}) = 0 THEN NULL
        ELSE
            aggregate(
            zip_with(
              {arr_col},
              sequence(0, size({arr_col}) - 1),
              (x, i) -> x * pow({decay}, (size({arr_col}) - 1 - i))
            ),
            CAST(0.0 AS DOUBLE),
            (acc, v) -> acc + v
          )
          /
          aggregate(
            sequence(0, size({arr_col}) - 1),
            CAST(0.0 AS DOUBLE),
            (acc, i) -> acc + pow({decay}, (size({arr_col}) - 1 - i))
          )
      END
    """)

spark = SparkSession.builder.appName("ETH_Indicators_SparkOnly_SMA_EMA_RSI_MACD
    ").getOrCreate()

dynamic_frame = glueContext.create_dynamic_frame.from_catalog(
    database="trade_data_eth_imat3a03",
    table_name="eth_silver"
)

df = (
    dynamic_frame
    .toDF()
```

```
    .select(
        "datetime",
        "symbol",
        "open",
        "high",
        "low",
        "close",
        "volume",
        "year",
    )
)

df = df.withColumn("datetime", F.to_timestamp("datetime")).filter(F.col("
    datetime").isNotNull())

df = df.filter(F.col("symbol") == F.lit(ETH_SYMBOL))

w_order = Window.orderBy("datetime")

w200 = Window.orderBy("datetime").rowsBetween(-199, 0)
df = df.withColumn("SMA_200", F.avg("close").over(w200))

wK50 = Window.orderBy("datetime").rowsBetween(-(K_EMA50 - 1), 0)

close_hist_50 = (
    F.sort_array(F.collect_list(F.struct(F.col("datetime").alias("k"), F.col("
    close").alias("v"))).over(wK50))
)
df = df.withColumn("close_hist_50", F.transform(close_hist_50, lambda x: x["v"
    ]))
df = df.withColumn("EMA_50", ema_truncated_from_array("close_hist_50", span=50)
    ).drop("close_hist_50")

df = df.withColumn("prev_close", F.lag("close").over(w_order))
df = df.withColumn("delta", F.col("close") - F.col("prev_close"))

df = df.withColumn("gain", F.when(F.col("delta") > 0, F.col("delta")).otherwise
    (F.lit(0.0)))
df = df.withColumn("loss", F.when(F.col("delta") < 0, -F.col("delta")).
    otherwise(F.lit(0.0)))

w14 = Window.orderBy("datetime").rowsBetween(-13, 0)
df = df.withColumn("avg_gain_14", F.avg("gain").over(w14))
df = df.withColumn("avg_loss_14", F.avg("loss").over(w14))

df = df.withColumn(
    "RSI_14",
    F.when(F.col("avg_loss_14") == 0, F.lit(100.0))
     .otherwise(100.0 - (100.0 / (1.0 + (F.col("avg_gain_14") / F.col("
    avg_loss_14")))))
)
```

```
wK12 = Window.orderBy("datetime").rowsBetween(-(K_EMA12 - 1), 0)
close_hist_12 = F.sort_array(F.collect_list(F.struct(F.col("datetime").alias("k
    "), F.col("close").alias("v"))).over(wK12))
df = df.withColumn("close_hist_12", F.transform(close_hist_12, lambda x: x["v"
    ]))
df = df.withColumn("EMA_12", ema_truncated_from_array("close_hist_12", span=12)
    ).drop("close_hist_12")

wK26 = Window.orderBy("datetime").rowsBetween(-(K_EMA26 - 1), 0)
close_hist_26 = F.sort_array(F.collect_list(F.struct(F.col("datetime").alias("k
    "), F.col("close").alias("v"))).over(wK26))
df = df.withColumn("close_hist_26", F.transform(close_hist_26, lambda x: x["v"
    ]))
df = df.withColumn("EMA_26", ema_truncated_from_array("close_hist_26", span=26)
    ).drop("close_hist_26")

df = df.withColumn("MACD", F.col("EMA_12") - F.col("EMA_26"))

df = df.drop("prev_close", "delta", "gain", "loss", "avg_gain_14", "avg_loss_14
    ", "EMA_12", "EMA_26")

df.write.mode("overwrite").partitionBy("year").parquet(OUTPUT_PATH)

spark.stop()
```

Una vez calculadas estas métricas para cada día del dataset, los resultados se almacenan en el bucket común, dentro de la carpeta `gold/`.

Finalmente, ejecutamos de nuevo el crawler para actualizar el catálogo y poder realizar consultas sobre todas las capas (tablas) utilizando Athena.

2.1.3. Evidencias

Aquí se muestran algunas de las evidencias del éxito de la implementación descrita anteriormente:

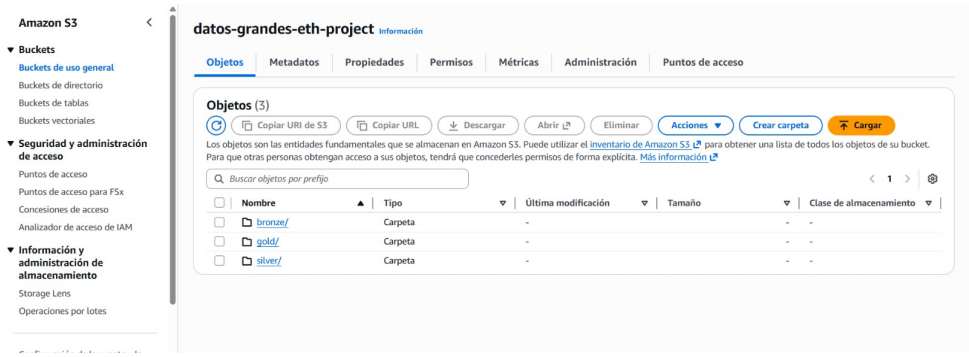

Figura 1: Archivos guardados correctamente en S3

Resultados (10)

Copiar | Descargar resultados en formato CSV

Filas de búsqueda

#	datetime	symbol	open	high	low	close	volume	partition_0	year
1	2026-01-01 01:00:00	BINANCE:ETHUSDT	2971.65	3010.0	2971.15	3004.19	136088.5207	bronze	2026
2	2026-01-02 01:00:00	BINANCE:ETHUSDT	3004.19	3150.01	2992.49	3125.45	369891.2449	bronze	2026
3	2026-01-03 01:00:00	BINANCE:ETHUSDT	3125.46	3136.71	3076.0	3127.11	134405.2258	bronze	2026
4	2026-01-04 01:00:00	BINANCE:ETHUSDT	3127.11	3167.22	3118.75	3144.7	181576.0686	bronze	2026
5	2026-01-05 01:00:00	BINANCE:ETHUSDT	3144.71	3265.66	3135.02	3224.99	382446.6618	bronze	2026
6	2026-01-06 01:00:00	BINANCE:ETHUSDT	3224.99	3308.86	3183.27	3296.84	412503.5066	bronze	2026
7	2026-01-07 01:00:00	BINANCE:ETHUSDT	3296.83	3296.84	3125.42	3168.72	321350.1225	bronze	2026
8	2026-01-08 01:00:00	BINANCE:ETHUSDT	3168.72	3184.39	3054.65	3106.65	343616.1005	bronze	2026
9	2026-01-09 01:00:00	BINANCE:ETHUSDT	3106.66	3148.41	3058.69	3088.4	274587.4533	bronze	2026

Figura 2: Datos curados (silver)

	open	high	low	close	volume	sma_200	ema_50	rsi_14
SDT	3337.78	3374.85	3313.88	3360.38	190238.5861	2985.032599999999	3414.991078163943	36.73286456984641
SDT	3360.39	3509.99	3354.5	3455.67	338632.8874	2984.188899999999	3416.586227113115	52.17298439678043
SDT	3455.68	3630.0	3423.44	3609.01	334041.0604	2984.6766499999994	3424.1321861668507	56.92517971043841
SDT	3609.0	3671.6	3572.42	3656.88	229266.3133	2985.543949999999	3433.2594981082634	67.61980759852818
SDT	3656.88	3675.25	3593.7	3635.99	155444.1741	2985.9213499999996	3441.2096107117554	70.44614816183261
SDT	3636.0	3744.83	3610.63	3687.45	329642.3648	2986.7931999999987	3450.86602619833	67.05113021986507
SDT	3687.44	3700.86	3356.31	3381.31	541543.2874	2986.1072499999987	3448.1381889085465	44.47462783874824
SDT	3381.31	3415.1	3208.2	3327.29	584749.9627	2985.264949999999	3443.398910873449	42.01387254948095
SDT	3327.29	3357.27	3158.0	3219.2	501818.4247	2984.2563999999984	3434.606670908106	44.25728930175377
SDT	3219.2	3322.49	3193.97	3267.04	454142.1333	2983.8279499999985	3428.035367314869	46.84967345042986

Figura 3: Datos con KPIs (gold)

2.1.4. Resultado

Aquí se muestra un ejemplo de una línea oro:

```
{
  datetime: "2024-01-01 01:00:00.000",
  symbol: "BINANCE:ETHUSDT",
  open: 2281.87,
  high: 2352.37,
  low: 2265.24,
  close: 2352.04,
  volume: 216702.6914,
  partition_0: "gold",
  sma_200: 1864.306050000001,
  ema_50: 2171.7241774326885,
  rsi_14: 60.860236188229926,
  macd: 39.37771473472321,
  year: 2024
}
```

3. Arquitectura del Sprint

En este sprint se desarrolla primero el job `bronze2silver`, que transforma los datos crudos en formato Parquet. Posteriormente, se implementa y ejecuta el job `silver2gold`, encargado de calcular los KPIs técnicos. Finalmente, se ejecuta el crawler para actualizar el catálogo y permitir consultas sobre todas las capas desde Athena.

La arquitectura del sprint queda resumida en el siguiente flujo:

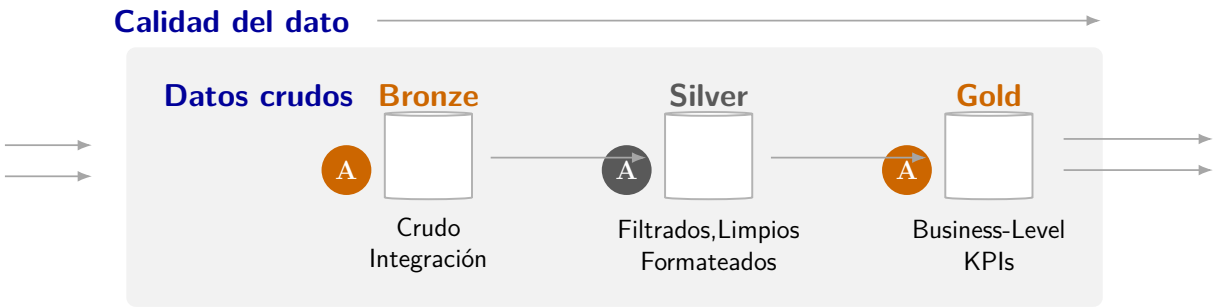

Figura 4: Arquitectura Bronze/Silver/Gold para mejora de calidad del dato

4. Validación y Pruebas

Una vez completado el proceso, podemos realizar consultas de forma sencilla sobre los KPIs calculados utilizando Athena.

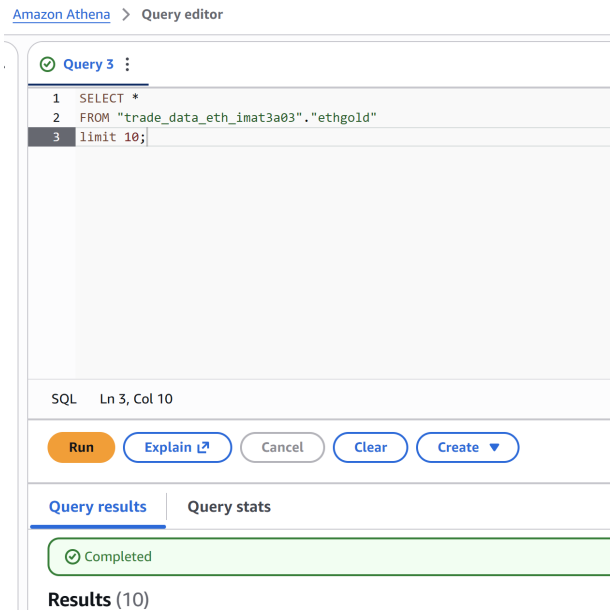

Figura 5: Query exitosa en Athena

5. Conclusiones del Sprint

Tras este sprint, no solo disponemos de los datos estructurados en distintas capas, sino también de indicadores técnicos que nos permiten extraer *insights* directamente sobre la

evolución de la criptomoneda. Además, la arquitectura implementada facilita consultas eficientes y escalables sobre la información.